

DJANGO

Introduction to Django

- Django is a Python-based web framework that allows you to quickly create efficient web applications.
- Django provides built-in features for everything including Django Admin Interface, default database – SQLite3, etc.
- When you're building a website, you always need a similar set of components: a way to handle user authentication (signing up, signing in, signing out), a management panel for your website, forms, a way to upload files, etc. Django gives you ready-made components to use and that too for rapid development.

How does Django Work?

Django follows the MVT design pattern (Model View Template).

- Model - The data you want to present, usually data from a database.
- View - A request handler that returns the relevant template and content - based on the request from the user.
- Template - A text file (like an HTML file) containing the layout of the web page, with logic on how to display the data.

Model

The model provides data from the database.

In Django, the data is delivered as an Object Relational Mapping (ORM), which is a technique designed to make it easier to work with databases.

The most common way to extract data from a database is SQL. One problem with SQL is that you have to have a pretty good understanding of the database structure to be able to work with it.

Django, with ORM, makes it easier to communicate with the database, without having to write complex SQL statements.

The models are usually located in a file called `models.py`

View

A view is a function or method that takes http requests as arguments, imports the relevant model(s), and finds out what data to send to the template, and returns the final result.

The views are usually located in a file called **views.py**.

Template

A template is a file where you describe how the result should be represented.

Templates are often .html files, with HTML code describing the layout of a web page, but it can also be in other file formats to present other results, but we will concentrate on .html files.

Django uses standard HTML to describe the layout, but uses Django tags to add logic:

```
<h1>My Homepage</h1> <p>My name is {{ firstname }}.</p>
```

The templates of an application is located in a folder named **templates**.

URLs

Django also provides a way to navigate around the different pages in a website.

When a user requests a URL, Django decides which *view* it will send it to.
This is done in a file called `urls.py`.

How does everything work?

When you have installed Django and created your first Django web application, and the browser requests the URL, this is basically what happens:

1. Django receives the URL, checks the `urls.py` file, and calls the view that matches the URL.
2. The view, located in `views.py`, checks for relevant models.
3. The models are imported from the `models.py` file.
4. The view then sends the data to a specified template in the `template` folder.
5. The template contains HTML and Django tags, and with the data it returns finished HTML content back to the browser.

Create a new folder Django

Open Anaconda Command prompt & change directory to Django folder

Step1:Create Virtual environment

conda create --name myenv python==3.9

Step2: Activate Virtual environment

conda activate myenv

Step 3: Install Django

```

Select C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19045.3208]
(c) Microsoft Corporation. All rights reserved.

D:\django>python -m venv myenv

D:\django>dir
Volume in drive D has no label.
Volume Serial Number is 6E3F-C723

Directory of D:\django

24-07-2023  12:39    <DIR>          .
24-07-2023  12:39    <DIR>          ..
24-07-2023  12:39    <DIR>          myenv
               0 File(s)                0 bytes
               3 Dir(s)  80,332,935,168 bytes free

D:\django>myenv\scripts\activate

(myenv) D:\django>python -m pip install django
```

```
(myenv) D:\django>python -m pip install django
Collecting django
  Downloading Django-4.2.3-py3-none-any.whl (8.0 MB)
----- 8.0/8.0 MB 5.5 MB/s eta 0:00:00
Collecting asgiref<4,>=3.6.0
  Downloading asgiref-3.7.2-py3-none-any.whl (24 kB)
Collecting sqlparse>=0.3.1
  Using cached sqlparse-0.4.4-py3-none-any.whl (41 kB)
Collecting tzdata
  Using cached tzdata-2023.3-py2.py3-none-any.whl (341 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.7.2 django-4.2.3 sqlparse-0.4.4 tzdata-2023.3

[notice] A new release of pip available: 22.3.1 -> 23.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip

(myenv) D:\django>
```

Step 4: create django project

```
(myenv) D:\django>django-admin startproject djproject  
  
(myenv) D:\django>
```

```
(myenv) D:\django>dir  
Volume in drive D has no label.  
Volume Serial Number is 6E3F-C723  
  
Directory of D:\django  
  
24-07-2023  12:45    <DIR>          .  
24-07-2023  12:45    <DIR>          ..  
24-07-2023  12:45    <DIR>          djproject  
24-07-2023  12:39    <DIR>          myenv  
                0 File(s)                0 bytes  
                4 Dir(s)  80,284,409,856 bytes free  
  
(myenv) D:\django>
```

Step 5: Change directory to the project folder

```
(myenv) D:\django>cd djproject

(myenv) D:\django\djproject>dir
Volume in drive D has no label.
Volume Serial Number is 6E3F-C723

Directory of D:\django\djproject

24-07-2023  12:45    <DIR>          .
24-07-2023  12:45    <DIR>          ..
24-07-2023  12:45    <DIR>          djproject
24-07-2023  12:45                687 manage.py
                1 File(s)                687 bytes
                3 Dir(s)  80,284,409,856 bytes free
```

Step 6: Run the django server

```
(myenv) D:\django\djproject>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(
s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
July 24, 2023 - 12:50:28
Django version 4.2.3, using settings 'djproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

The install worked successfully! ✕

127.0.0.1:8000

↶ ↷ ↻ 🔍 🌟 📦 D ⋮

django

View [release notes](#) for Django 4.2



The install worked successfully! Congratulations!

You are seeing this page because `DEBUG=True` is in your settings file and you have not configured any URLs.

 **Django Documentation**
Topics, references, & how-to's

 **Tutorial: A Polling App**
Get started with Django

 **Django Community**
Connect, get help, or contribute





 26°C



ENG

12:51
24-07-2023

Step 7: Create an app

```
(myenv) D:\django\django>python manage.py startapp userapp  
(myenv) D:\django\django>
```

```
(myenv) D:\django\django>python manage.py startapp userapp  
(myenv) D:\django\django>dir  
Volume in drive D has no label.  
Volume Serial Number is 6E3F-C723  
  
Directory of D:\django\django  
  
24-07-2023  12:54    <DIR>          .  
24-07-2023  12:54    <DIR>          ..  
24-07-2023  12:50             0 db.sqlite3  
24-07-2023  12:50    <DIR>          django  
24-07-2023  12:45             687 manage.py  
24-07-2023  12:54    <DIR>          userapp  
                2 File(s)              687 bytes  
                4 Dir(s)  80,283,693,056 bytes free
```

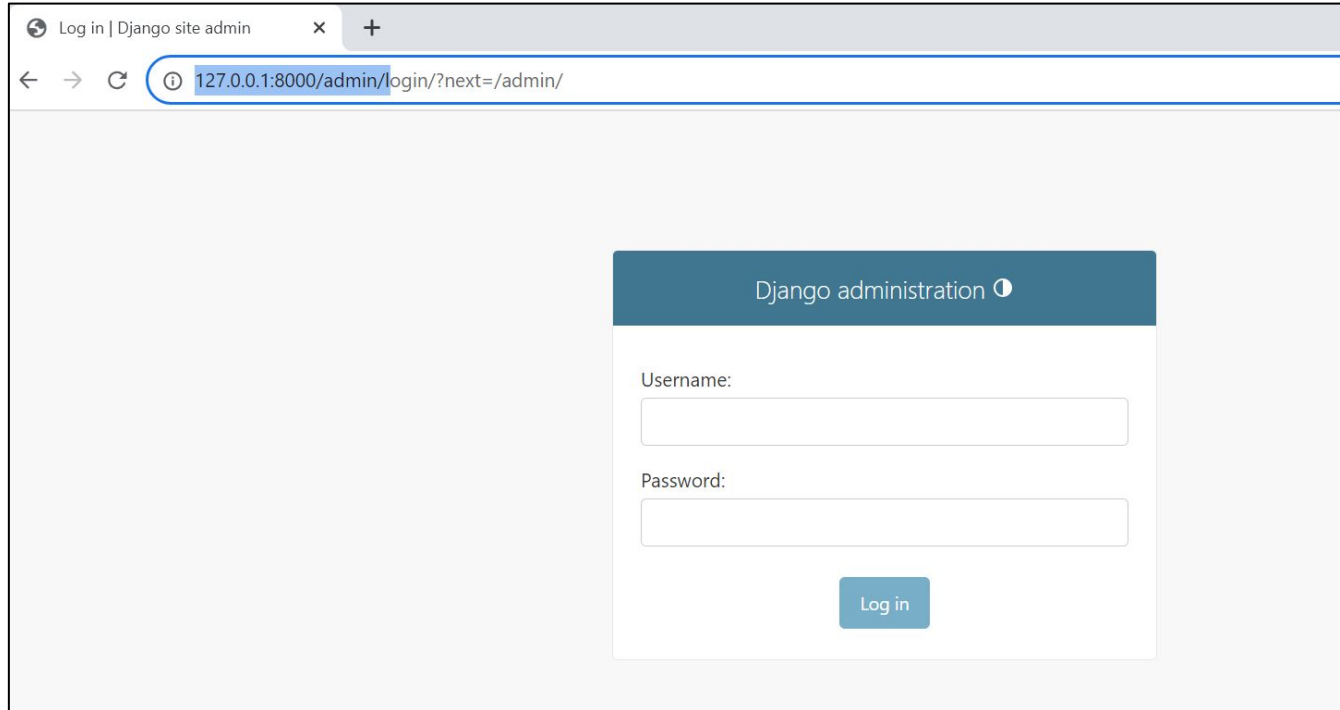
Step 8: Create superuser

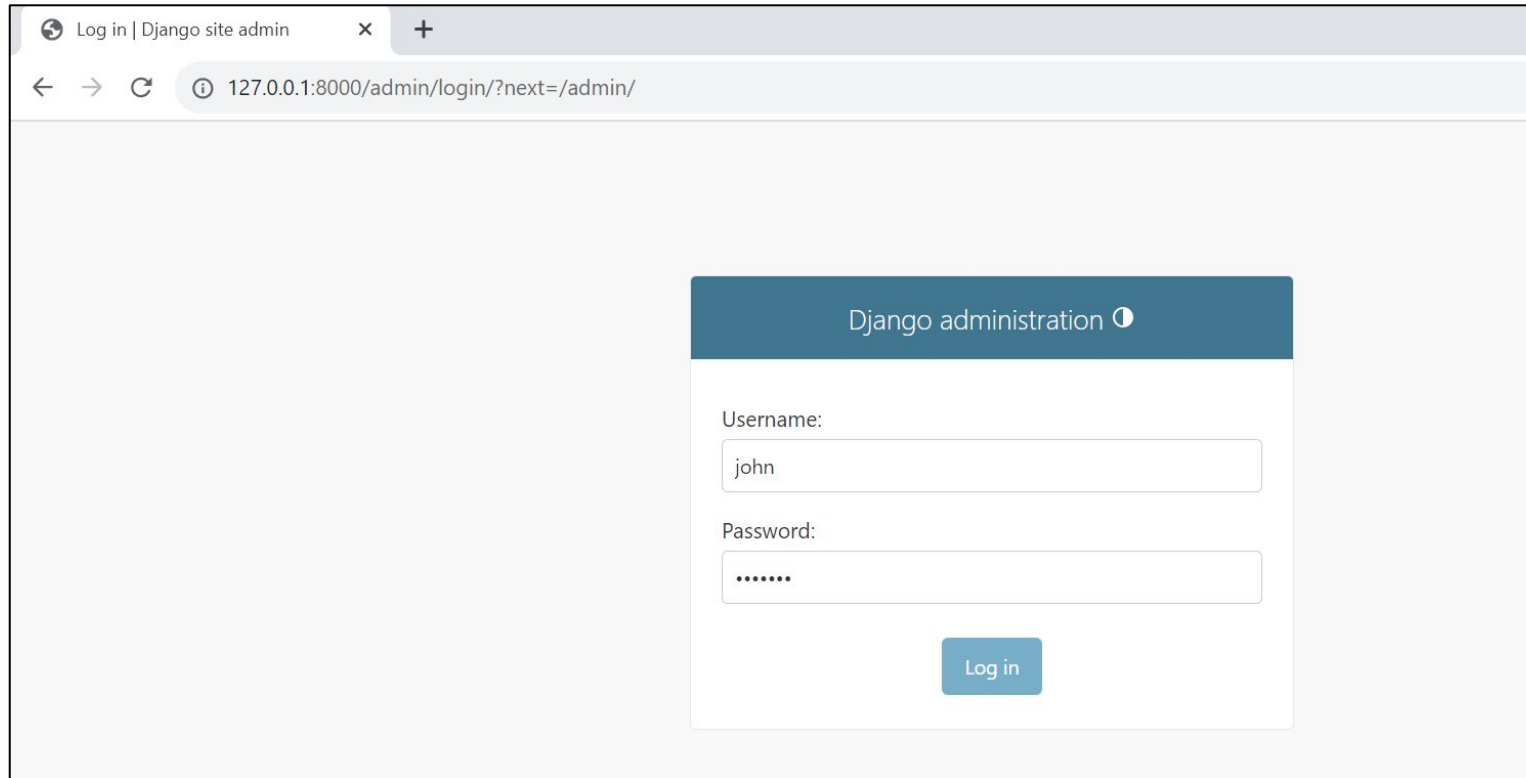
```
(myenv) D:\django\djproject>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(
s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
July 24, 2023 - 12:57:29
Django version 4.2.3, using settings 'djproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Paste url on browser and change url to `http://127.0.0.1:8000/admin`





The image shows a web browser window with a single tab titled "Log in | Django site admin". The address bar displays the URL "127.0.0.1:8000/admin/login/?next=/admin/". The main content area features a login form titled "Django administration" in a dark blue header. Below the header, the form contains two input fields: "Username:" with the value "john" and "Password:" with masked characters ".....". A blue "Log in" button is positioned at the bottom right of the form.

Log in | Django site admin

127.0.0.1:8000/admin/login/?next=/admin/

Django administration

Username:

john

Password:

.....

Log in

OperationalError at /admin/login/

no such table: auth_user

Request Method: POST

Request URL: http://127.0.0.1:8000/admin/login/?next=/admin/

Django Version: 4.2.3

Exception Type: OperationalError

Exception Value: no such table: auth_user

Exception Location: D:\django\myenv\Lib\site-packages\django\db\backends\sqlite3\base.py, line 328, in execute

Raised during: django.contrib.admin.sites.login

Python Executable: D:\django\myenv\Scripts\python.exe

Python Version: 3.11.3

Python Path: ['D:\\django\\djproject',
'C:\\Users\\Datamites\\AppData\\Local\\Programs\\Python\\Python311\\python311.zip',
'C:\\Users\\Datamites\\AppData\\Local\\Programs\\Python\\Python311\\DLLs',
'C:\\Users\\Datamites\\AppData\\Local\\Programs\\Python\\Python311\\Lib',
'C:\\Users\\Datamites\\AppData\\Local\\Programs\\Python\\Python311',
'D:\\django\\myenv',
'D:\\django\\myenv\\Lib\\site-packages']

Server time: Mon, 24 Jul 2023 07:33:16 +0000

Traceback [Switch to copy-and-paste view](#)

D:\django\myenv\Lib\site-packages\django\db\backends\utils.py, line 89, in _execute

```
89.         return self.cursor.execute(sql, params)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

► Local vars

D:\django\myenv\Lib\site-packages\django\db\backends\sqlite3\base.py, line 328, in execute

Create superuser

```
(myenv) D:\django\djproject>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
  Applying admin.0003_logentry_add_action_flag_choices... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0003_alter_user_email_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK
  Applying auth.0009_alter_user_last_name_max_length... OK
  Applying auth.0010_alter_group_name_max_length... OK
  Applying auth.0011_update_proxy_permissions... OK
  Applying auth.0012_alter_user_first_name_max_length... OK
  Applying sessions.0001_initial... OK
```

```
(myenv) D:\django\djproject>python manage.py createsuperuser
Username (leave blank to use 'datamites'): datamites
Email address: data@gmail.com
Password:
Password (again):
The password is too similar to the username.
Bypass password validation and create user anyway? [y/N]: y
Superuser created successfully.

(myenv) D:\django\djproject>
```

```
(myenv) D:\django\djproject>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
July 24, 2023 - 13:07:26
Django version 4.2.3, using settings 'djproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Log in | Django site admin

127.0.0.1:8000/admin/login/?next=/admin/

Django administration ⓘ

Username:

Password:

Log in

Site administration | Django site x +

127.0.0.1:8000/admin/

Django administration

WELCOME, **DATAMITES**. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Site administration

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#) [Change](#)

Users [+ Add](#) [Change](#)

Recent actions

My actions

None available

Select user to change | Django site x +

127.0.0.1:8000/admin/auth/user/

Django administration

WELCOME, **DATAMITES**. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Authentication and Authorization > Users

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Users [+ Add](#)

Select user to change

Search

Action: Go 0 of 1 selected

☐	USERNAME	EMAIL ADDRESS	FIRST NAME	LAST NAME	STAFF STATUS
☐	datamites	data@gmail.com			☒

1 user

FILTER

↓ By staff status

All

Yes

No

↓ By superuser status

All

Yes

No

[ADD USER](#) +

Django administration

WELCOME, **DATAMITES**. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Home > Authentication and Authorization > Users > Add user

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#)

Users [+ Add](#)

Add user

First, enter a username and password. Then, you'll be able to edit more user options.

Username:

akshatha

Required. 150 characters or fewer. Letters, digits and @/./+/-/_ only.

Password:

.....

Your password can't be too similar to your other personal information.

Your password must contain at least 8 characters.

Your password can't be a commonly used password.

Your password can't be entirely numeric.

Password confirmation:

.....

Enter the same password as before, for verification.

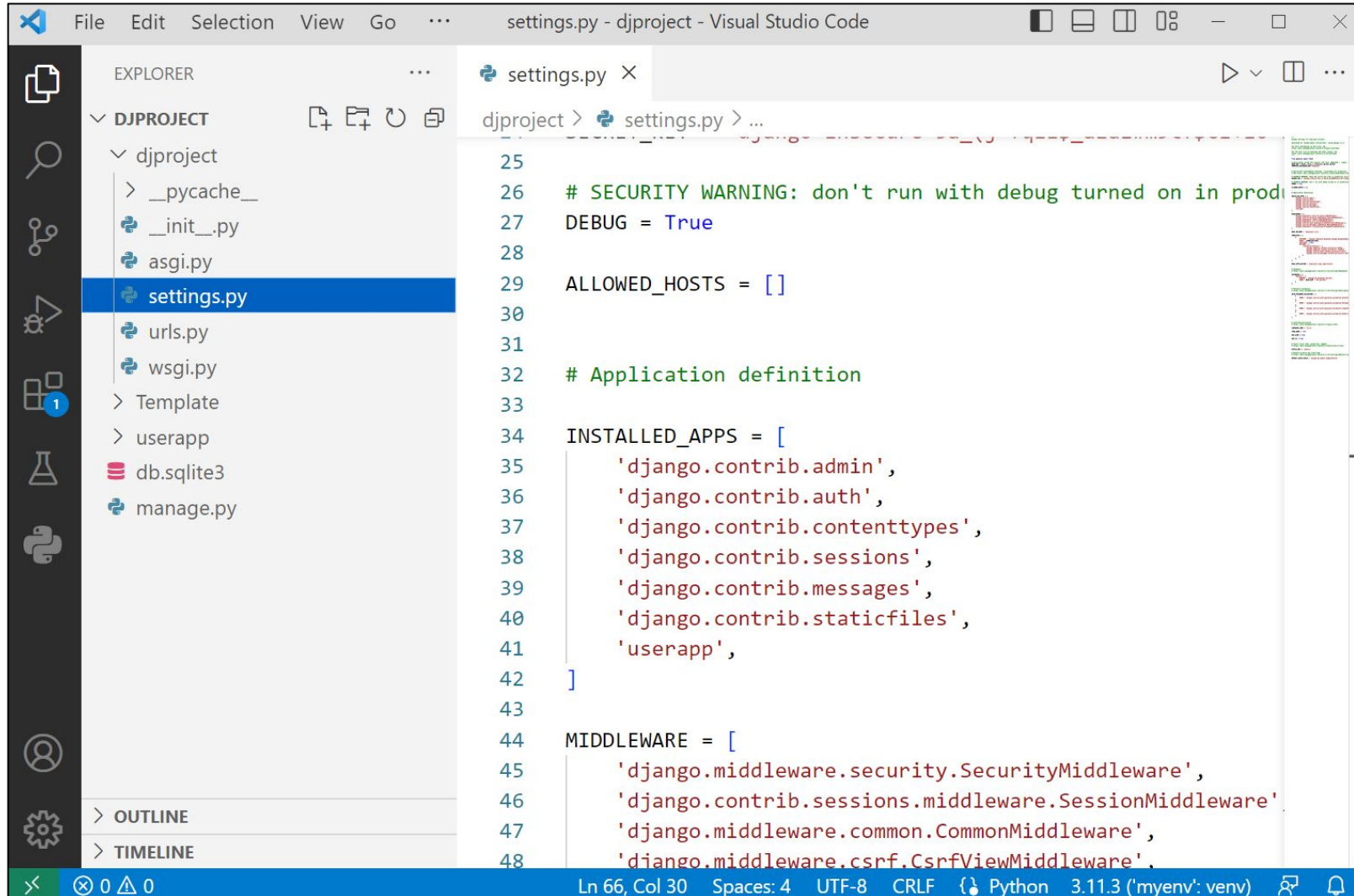
SAVE

Save and add another

Save and continue editing

Creating Virtual Environment: Activate Virtual Environment:	conda create --name myenv python==3.9
	conda activate myenv
Install DJANGO	pip install django
project	django-admin startproject project_name
Change directory	cd project_name
Runserver	python manage.py runserver
Migrate	Python manage.py migrate
Superuser	python manage.py createsuperuser
Provide	Username and password
Runserver	python manage.py runserver

Update App name in settings.py



```
File Edit Selection View Go ... settings.py - djproject - Visual Studio Code
EXPLORER
  DJPROJECT
    djproject
      __pycache__
      __init__.py
      asgi.py
      settings.py
      urls.py
      wsgi.py
      Template
      userapp
      db.sqlite3
      manage.py
  OUTLINE
  TIMELINE

settings.py
25
26 # SECURITY WARNING: don't run with debug turned on in production
27 DEBUG = True
28
29 ALLOWED_HOSTS = []
30
31
32 # Application definition
33
34 INSTALLED_APPS = [
35     'django.contrib.admin',
36     'django.contrib.auth',
37     'django.contrib.contenttypes',
38     'django.contrib.sessions',
39     'django.contrib.messages',
40     'django.contrib.staticfiles',
41     'userapp',
42 ]
43
44 MIDDLEWARE = [
45     'django.middleware.security.SecurityMiddleware',
46     'django.contrib.sessions.middleware.SessionMiddleware',
47     'django.middleware.common.CommonMiddleware',
48     'django.middleware.csrf.CsrfViewMiddleware',
```

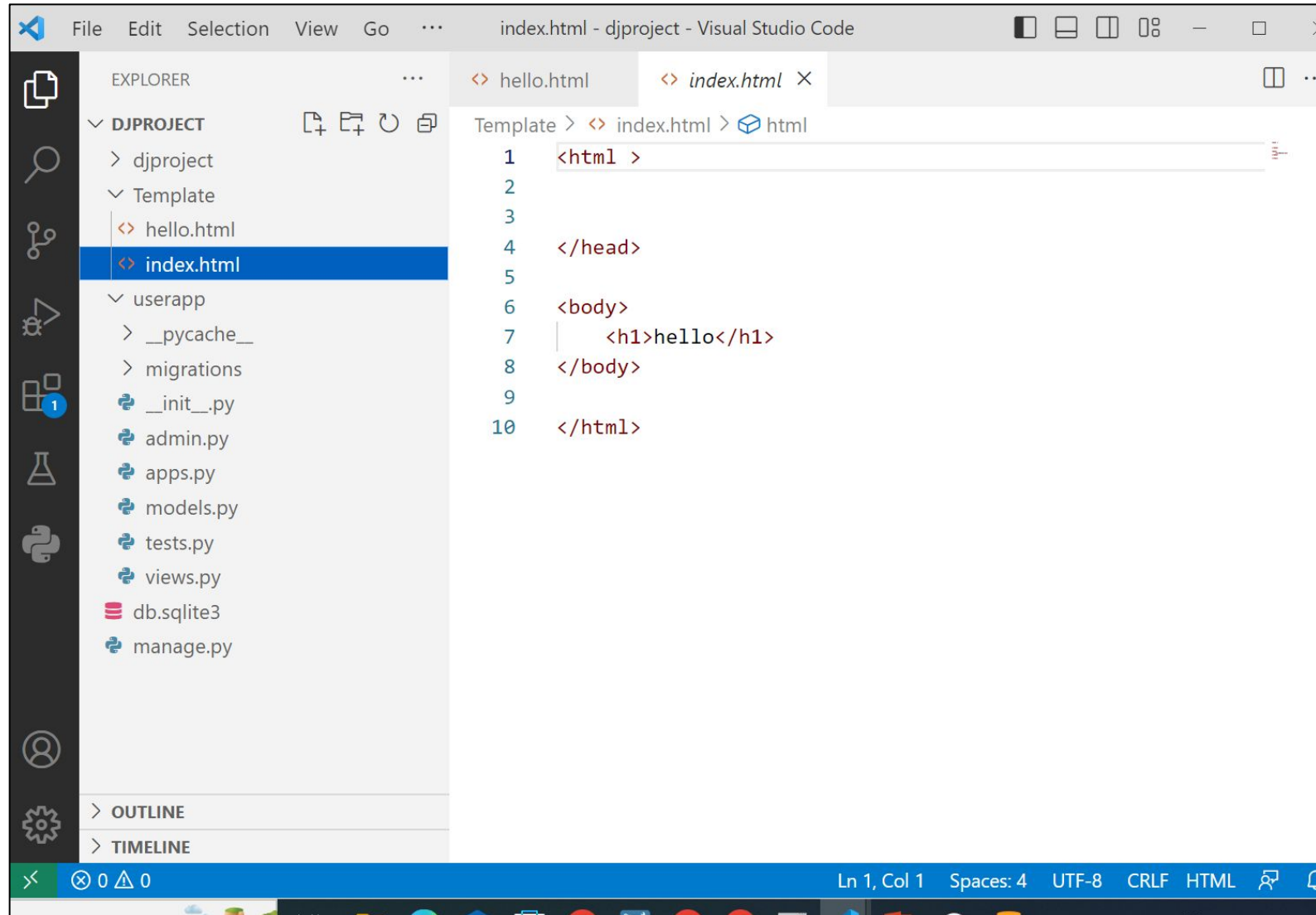
Create a template folder and update path in settings.py

django > settings.py > ...

```
1  """
2  Django settings for django project.
3
4  Generated by 'django-admin startproject' using Django 4.2.3.
5
6  For more information on this file, see
7  https://docs.djangoproject.com/en/4.2/topics/settings/
8
9  For the full list of settings and their values, see
10 https://docs.djangoproject.com/en/4.2/ref/settings/
11 """
12
13 from pathlib import Path
14
15 # Build paths inside the project like this: BASE_DIR / 'subdir'.
16 BASE_DIR = Path(__file__).resolve().parent.parent
17 TEMPLATE_DIR=BASE_DIR/"Template"
```

```
53
54 ROOT_URLCONF = 'django.urls'
55
56 TEMPLATES = [
57     {
58         'BACKEND': 'django.template.backends.django.DjangoTemplates',
59         'DIRS': [TEMPLATE_DIR],
60         'APP_DIRS': True,
61         'OPTIONS': {
62             'context_processors': [
63                 'django.template.context_processors.debug',
64                 'django.template.context_processors.request',
65                 'django.contrib.auth.context_processors.auth',
66                 'django.contrib.messages.context_processors.messages',
67             ],
68         },
69     ],
70 ]
```

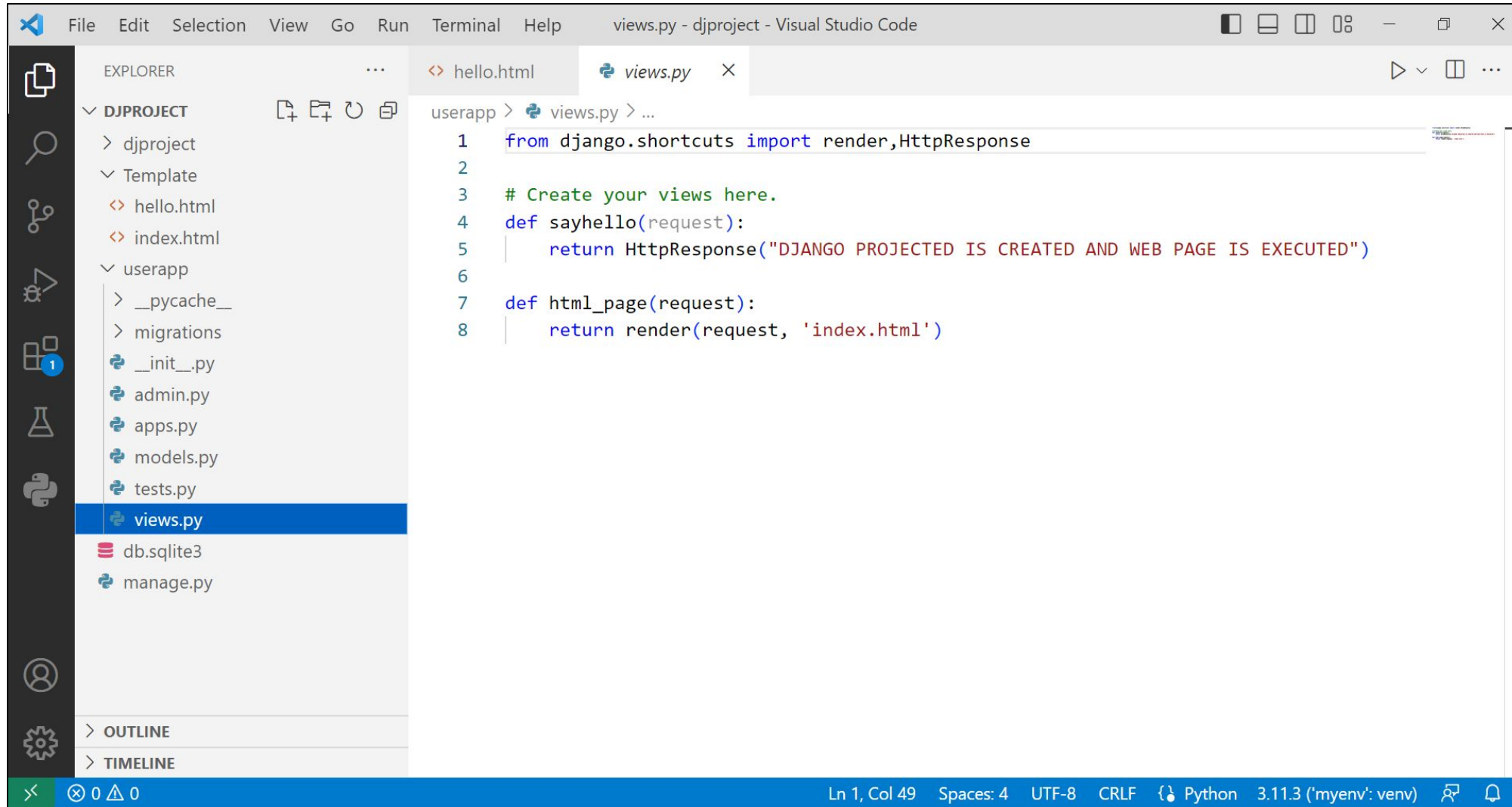
Create Html files in template folder



The screenshot shows the Visual Studio Code interface with a Django project named 'djproject'. The Explorer sidebar on the left displays the project structure, including a 'Template' folder. The 'index.html' file within the 'Template' folder is selected and highlighted in blue. The main editor window shows the content of 'index.html', which is a basic HTML template with a head section and a body section containing a single line of HTML: `<h1>hello</h1>`. The status bar at the bottom indicates the current position is Line 1, Column 1, with 4 spaces, using UTF-8 encoding, CRLF line endings, and HTML syntax.

```
1 <html>  
2  
3  
4 </head>  
5  
6 <body>  
7   <h1>hello</h1>  
8 </body>  
9  
10 </html>
```

Update views.py file



The screenshot shows the Visual Studio Code interface with a Django project named 'djproject'. The Explorer sidebar on the left displays the project structure, including 'djproject', 'Template', 'userapp', and various Python files. The 'views.py' file is selected and highlighted. The main editor window shows the code for 'views.py', which includes imports for 'render' and 'HttpResponse' from 'django.shortcuts', a comment '# Create your views here.', and two view functions: 'sayhello' and 'html_page'. The status bar at the bottom indicates the current line and column (Ln 1, Col 49), the number of spaces (4), the encoding (UTF-8), the line ending (CRLF), the Python version (3.11.3), and the environment (myenv: venv).

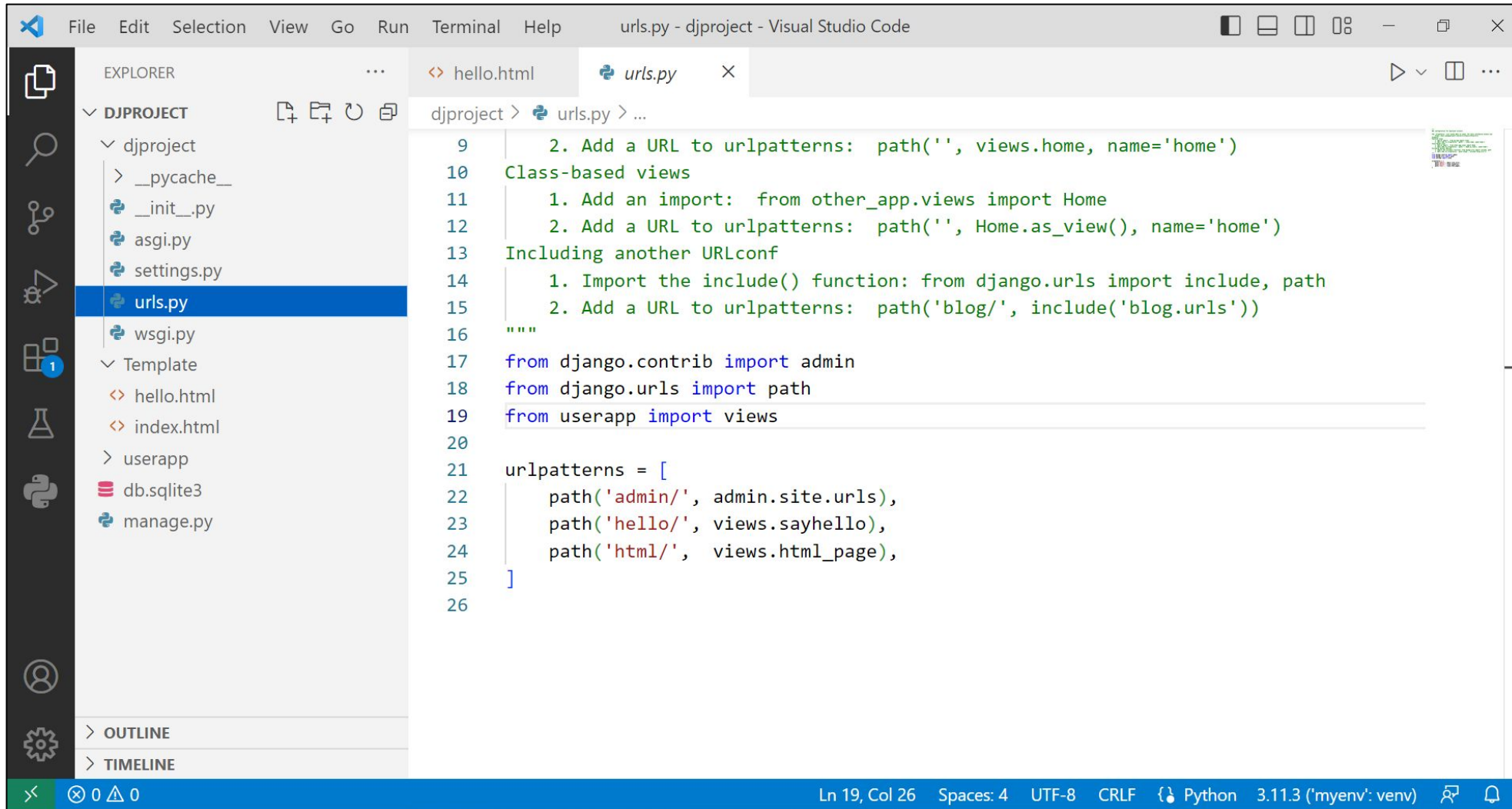
```
File Edit Selection View Go Run Terminal Help views.py - djproject - Visual Studio Code

EXPLORER
  DJPROJECT
    > djproject
    > Template
      <> hello.html
      <> index.html
    > userapp
      > __pycache__
      > migrations
      <+> __init__.py
      <+> admin.py
      <+> apps.py
      <+> models.py
      <+> tests.py
      <+> views.py
      db.sqlite3
      <+> manage.py
  > OUTLINE
  > TIMELINE

userapp > views.py > ...
1 from django.shortcuts import render,HttpResponse
2
3 # Create your views here.
4 def sayhello(request):
5     return HttpResponse("DJANGO PROJECTED IS CREATED AND WEB PAGE IS EXECUTED")
6
7 def html_page(request):
8     return render(request, 'index.html')
```

Ln 1, Col 49 Spaces: 4 UTF-8 CRLF Python 3.11.3 ('myenv': venv)

Update urls.py



The screenshot shows the Visual Studio Code interface with a Django project named 'djproject'. The Explorer sidebar on the left shows the project structure, with 'urls.py' selected under the 'djproject' folder. The main editor window displays the content of 'urls.py' with line numbers 9 through 26. The code includes comments for adding URLs to urlpatterns, imports for Django's admin and path functions, and a list of urlpatterns.

```
9      2. Add a URL to urlpatterns: path('', views.home, name='home')
10  Class-based views
11      1. Add an import: from other_app.views import Home
12      2. Add a URL to urlpatterns: path('', Home.as_view(), name='home')
13  Including another URLconf
14      1. Import the include() function: from django.urls import include, path
15      2. Add a URL to urlpatterns: path('blog/', include('blog.urls'))
16  """
17  from django.contrib import admin
18  from django.urls import path
19  from userapp import views
20
21  urlpatterns = [
22      path('admin/', admin.site.urls),
23      path('hello/', views.sayhello),
24      path('html/', views.html_page),
25  ]
26
```

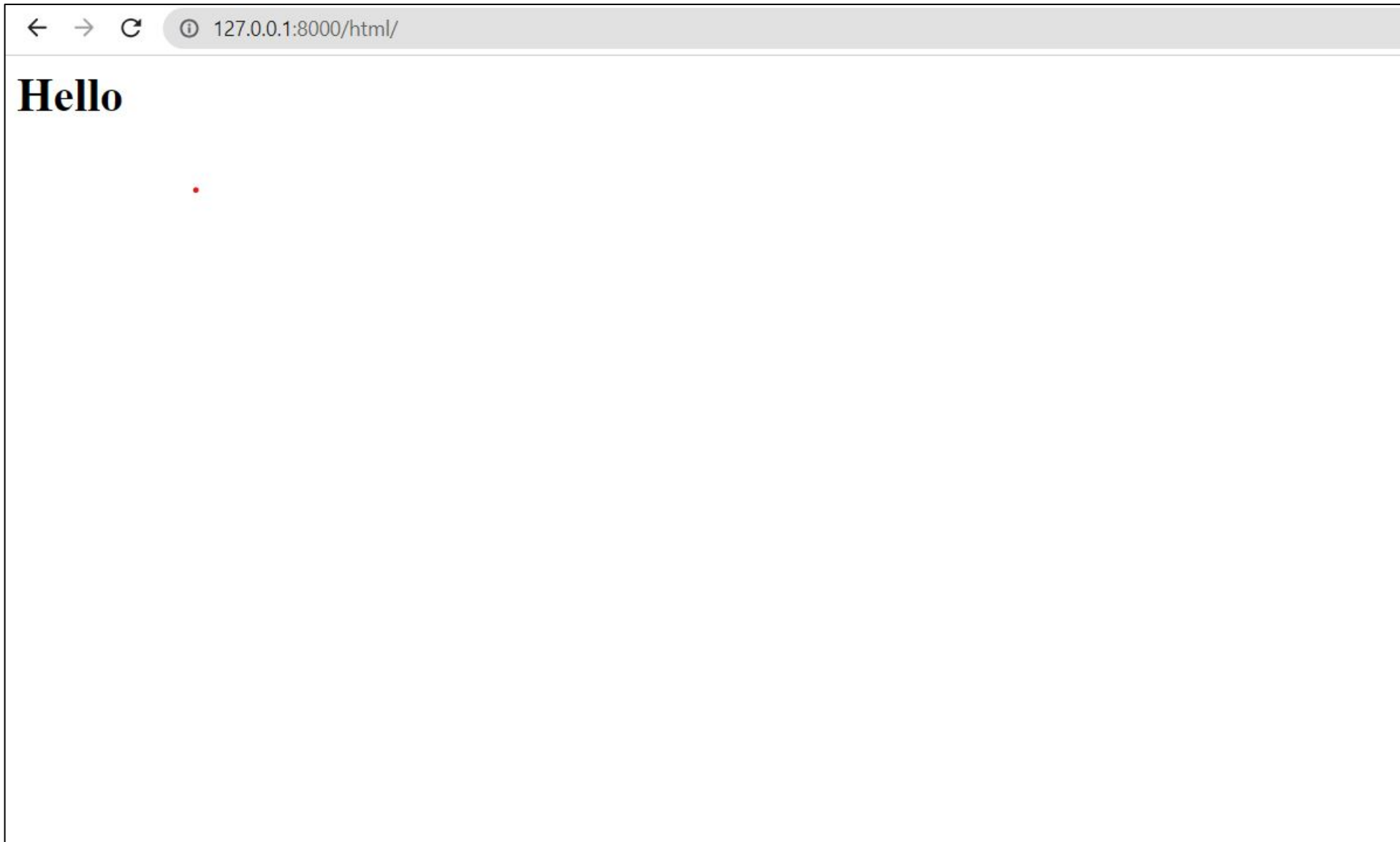
The status bar at the bottom indicates the current position is Line 19, Column 26, with 4 spaces, UTF-8 encoding, CRLF line endings, Python 3.11.3 environment, and a virtual environment named 'myenv'.

runserver

```
(myenv) D:\django\djproject>python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations for app(
s): admin, auth, contenttypes, sessions.
Run 'python manage.py migrate' to apply them.
July 24, 2023 - 12:50:28
Django version 4.2.3, using settings 'djproject.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```



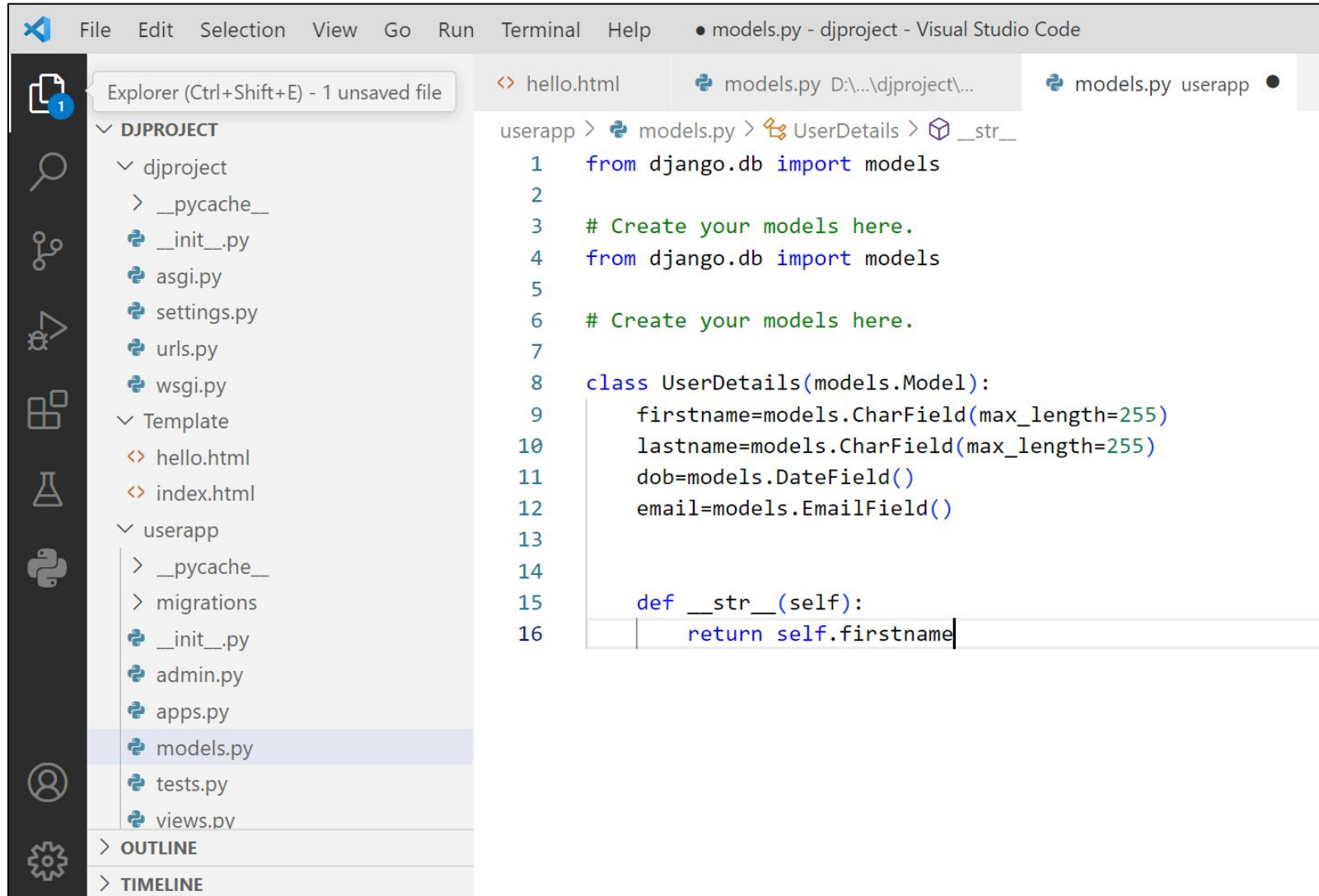
Django Models

Up until now, output has been static data from Python or HTML templates.

Now we will see how Django allows us to work with data, without having to change or upload files in the process.

In Django, data is created in objects, called Models, and is actually tables in a database.

Create Model (Table)



The screenshot shows the Visual Studio Code interface with a Django project named 'djproject'. The Explorer sidebar on the left shows the project structure, with the 'models.py' file in the 'userapp' directory selected. The main editor window displays the content of 'models.py', which includes imports for Django's models, a comment to create models, and a class definition for 'UserDetails' with fields for 'firstname', 'lastname', 'dob', and 'email'. The 'def __str__(self):' method is partially implemented, returning 'self.firstname'.

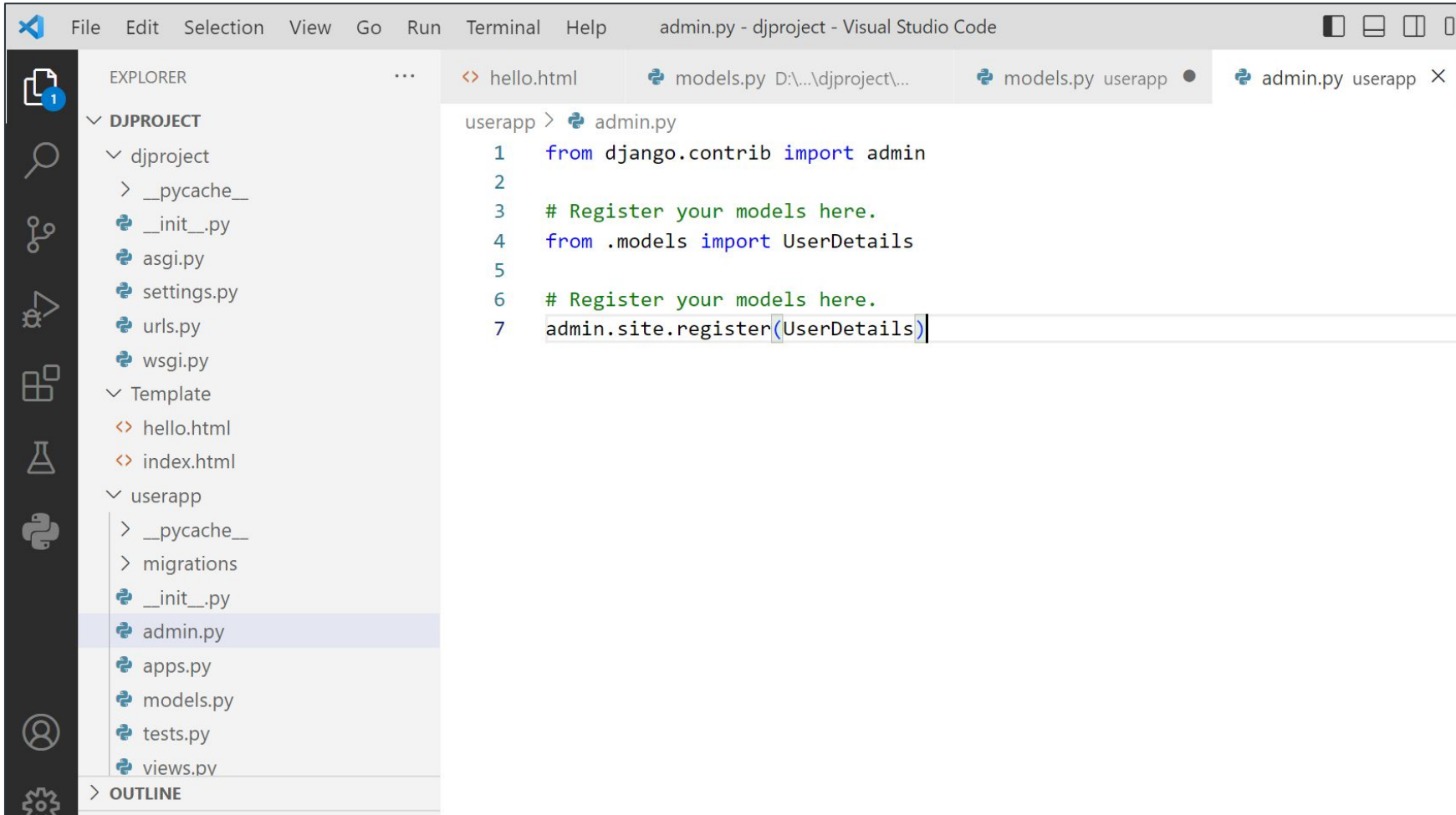
```
File Edit Selection View Go Run Terminal Help • models.py - djproject - Visual Studio Code

Explorer (Ctrl+Shift+E) - 1 unsaved file
D:\... \djproject\...
models.py userapp

DJPROJECT
├── djproject
│   ├── __pycache__
│   ├── __init__.py
│   ├── asgi.py
│   ├── settings.py
│   ├── urls.py
│   └── wsgi.py
├── Template
│   ├── hello.html
│   └── index.html
└── userapp
    ├── __pycache__
    ├── migrations
    ├── __init__.py
    ├── admin.py
    ├── apps.py
    ├── models.py
    ├── tests.py
    └── views.py

userapp > models.py > UserDetails > __str__
1  from django.db import models
2
3  # Create your models here.
4  from django.db import models
5
6  # Create your models here.
7
8  class UserDetails(models.Model):
9      firstname=models.CharField(max_length=255)
10     lastname=models.CharField(max_length=255)
11     dob=models.DateField()
12     email=models.EmailField()
13
14
15     def __str__(self):
16         return self.firstname
```

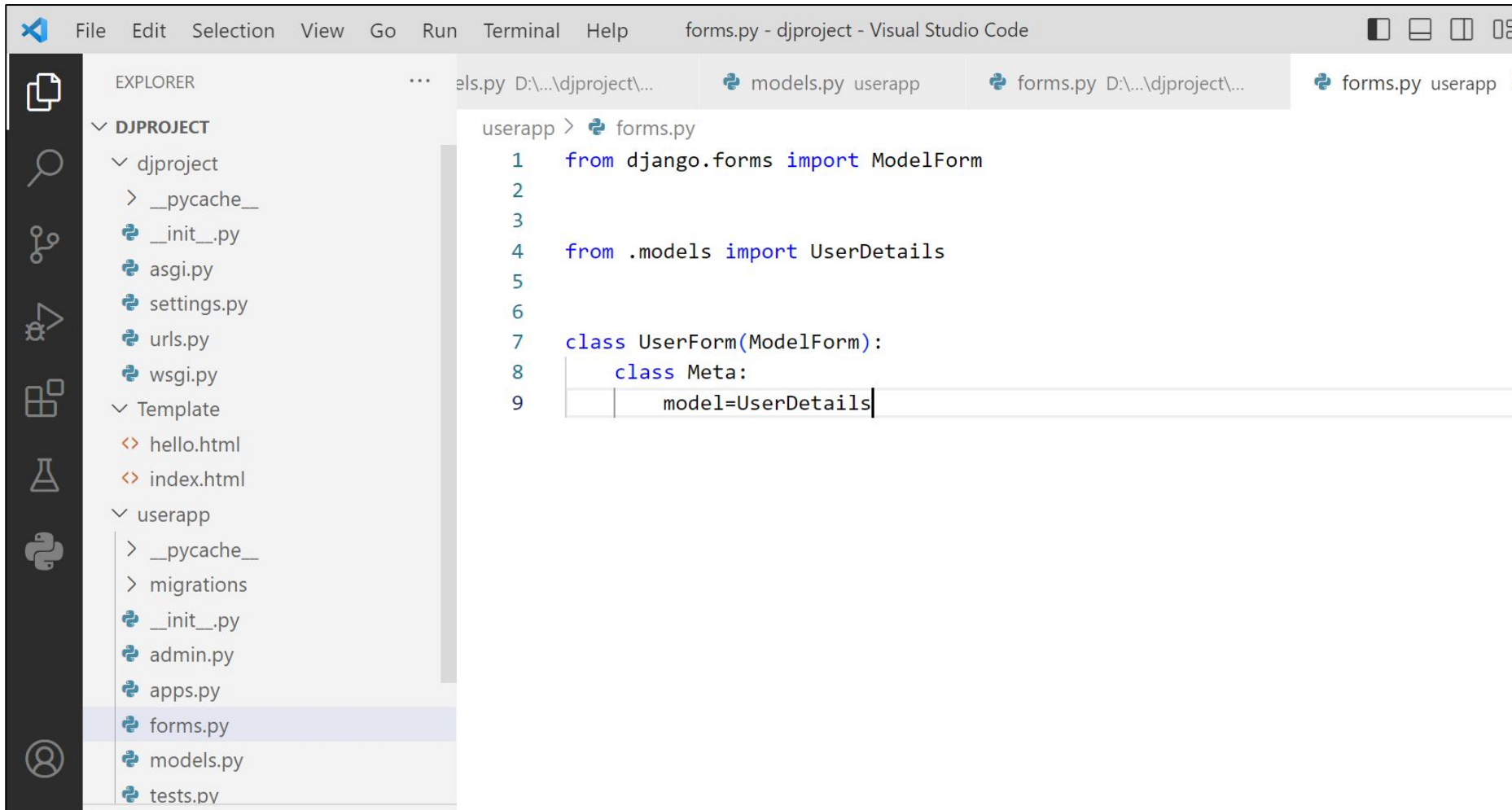
Register the model in admin.py



The screenshot shows the Visual Studio Code interface with the Django project 'djproject' open. The Explorer sidebar on the left shows the project structure, with 'admin.py' selected under the 'userapp' folder. The main editor window displays the content of 'admin.py' with the following code:

```
userapp > admin.py
1  from django.contrib import admin
2
3  # Register your models here.
4  from .models import UserDetails
5
6  # Register your models here.
7  admin.site.register(UserDetails)
```

Create forms.py in app



The screenshot shows the Visual Studio Code interface with the following components:

- Explorer Panel:** Displays the project structure. The `userapp` folder is expanded, showing files like `__pycache__`, `migrations`, `__init__.py`, `admin.py`, `apps.py`, `forms.py` (highlighted), `models.py`, and `tests.py`.
- Editor Panel:** Shows the content of `forms.py` in the `userapp` package. The code is as follows:

```
1 from django.forms import ModelForm
2
3
4 from .models import UserDetails
5
6
7 class UserForm(ModelForm):
8     class Meta:
9         model=UserDetails
```
- Tab Bar:** Shows four open tabs: `els.py`, `models.py userapp`, `forms.py D:\...\djproject\...`, and `forms.py userapp` (the active tab).

SQLite Database

When we created the Django project, we got an empty SQLite database.

It was created in the **django** root folder, and has the filename **db.sqlite3**.

By default, all Models created in the Django project will be created as tables in this database.

Migrate

Now when we have described a Model in the `models.py` file, we must run a command to actually create the table in the database.

```
(myenv) D:\django\djproject>python manage.py makemigrations
Migrations for 'userapp':
  userapp\migrations\0001_initial.py
    - Create model UserDetails

(myenv) D:\django\djproject>python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions, userapp
Running migrations:
  Applying userapp.0001_initial... OK

(myenv) D:\django\djproject>
```

Install **SQLite Viewer** extension to view SQL Database

The screenshot shows the Visual Studio Code interface with the SQLite Viewer extension installed. The Explorer sidebar on the left displays the project structure, with 'db.sqlite3' selected. The main editor area shows the SQLite Viewer interface, which includes a table list on the left and a table view on the right. The table view displays the following columns: id, firstname, lastname, and dob. The status bar at the bottom indicates 'Page 1 / 1' and 'Try SQLite Viewer W'.

id	firstname	lastname	dob
----	-----------	----------	-----

Runserver – ‘python manage.py runserver’ and login to admin user

The screenshot shows the Django administration interface in a web browser. The address bar indicates the URL is 127.0.0.1:8000/admin/. The page title is "Django administration". The main content area is titled "Site administration" and contains two sections: "AUTHENTICATION AND AUTHORIZATION" and "USERAPP". The "AUTHENTICATION AND AUTHORIZATION" section has two items: "Groups" and "Users", each with a green plus icon and the text "Add" and a yellow pencil icon and the text "Change". The "USERAPP" section has one item: "User detailss", also with a green plus icon and the text "Add" and a yellow pencil icon and the text "Change". On the right side, there is a "Recent actions" section with the heading "My actions" and two entries: "akshatha User" and "akshatha User", each with a yellow pencil icon.

127.0.0.1:8000/admin/

Django administration

WELCOME, **DATAMITES**. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#)

Site administration

AUTHENTICATION AND AUTHORIZATION

Groups [+ Add](#) [Change](#)

Users [+ Add](#) [Change](#)

USERAPP

User detailss [+ Add](#) [Change](#)

Recent actions

My actions

[akshatha](#)
User

[+ akshatha](#)
User

Django administration

WELCOME, **DATAMITES**. [VIEW SITE](#) / [CHANGE PASSWORD](#) / [LOG OUT](#) 

[Home](#) > [Userapp](#) > [User detailss](#) > [Add user details](#)

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

[Groups](#)  Add

[Users](#)  Add

USERAPP

[User detailss](#)  Add

Add user details

Firstname:

Lastname:

Dob:

 Today | 

Note: You are 5.5 hours ahead of server time.

Email:

SAVE

Save and add another

Save and continue editing

Django CRUD – Inserting, Updating and Deleting Data

```
(myenv) D:\django\djproject>python manage.py shell
Python 3.11.3 (tags/v3.11.3:f3909b8, Apr 4 2023, 23:49:59) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from userapp import models
>>> user=models.UserDetails(firstname="janvi",lastname="Reddy",dob="2022-09-08",email="abx@gmail.com")
>>> user.save()
>>>
```

```
>>> models.UserDetails.objects.all()
<QuerySet [<UserDetails: SHRAVYA>, <UserDetails: janvi>]>
>>> models.UserDetails.objects.get(id=2)
<UserDetails: janvi>
>>> user1= models.UserDetails.objects.get(id=2)
>>> user1.firstname="divya"
>>> user1.save()
>>> models.UserDetails.objects.all()
<QuerySet [<UserDetails: SHRAVYA>, <UserDetails: divya>]>
>>> user1= models.UserDetails.objects.get(id=2)
>>> user1.delete()
(1, {'userapp.UserDetails': 1})
>>> models.UserDetails.objects.all()
<QuerySet [<UserDetails: SHRAVYA>]>
>>>
```

Create Template all_users.html

- After creating Models, with the fields and data we want in them, it is time to display the data in a web page.
- Start by creating an HTML file named all_users.html and place it in the /templates/ folder:

The screenshot shows a code editor with a dark theme. On the left is the 'EXPLORER' sidebar showing a project structure. The main editor area displays the content of 'all_users.html'.

EXPLORER:

- DJPROJECT
 - ▼ djproject
 - > __pycache__
 - __init__.py
 - asgi.py
 - settings.py
 - urls.py
 - wsgi.py
 - ▼ Template
 - <> all_users.html
 - <> index.html
 - ▼ userapp
 - > __pycache__
 - > migrations

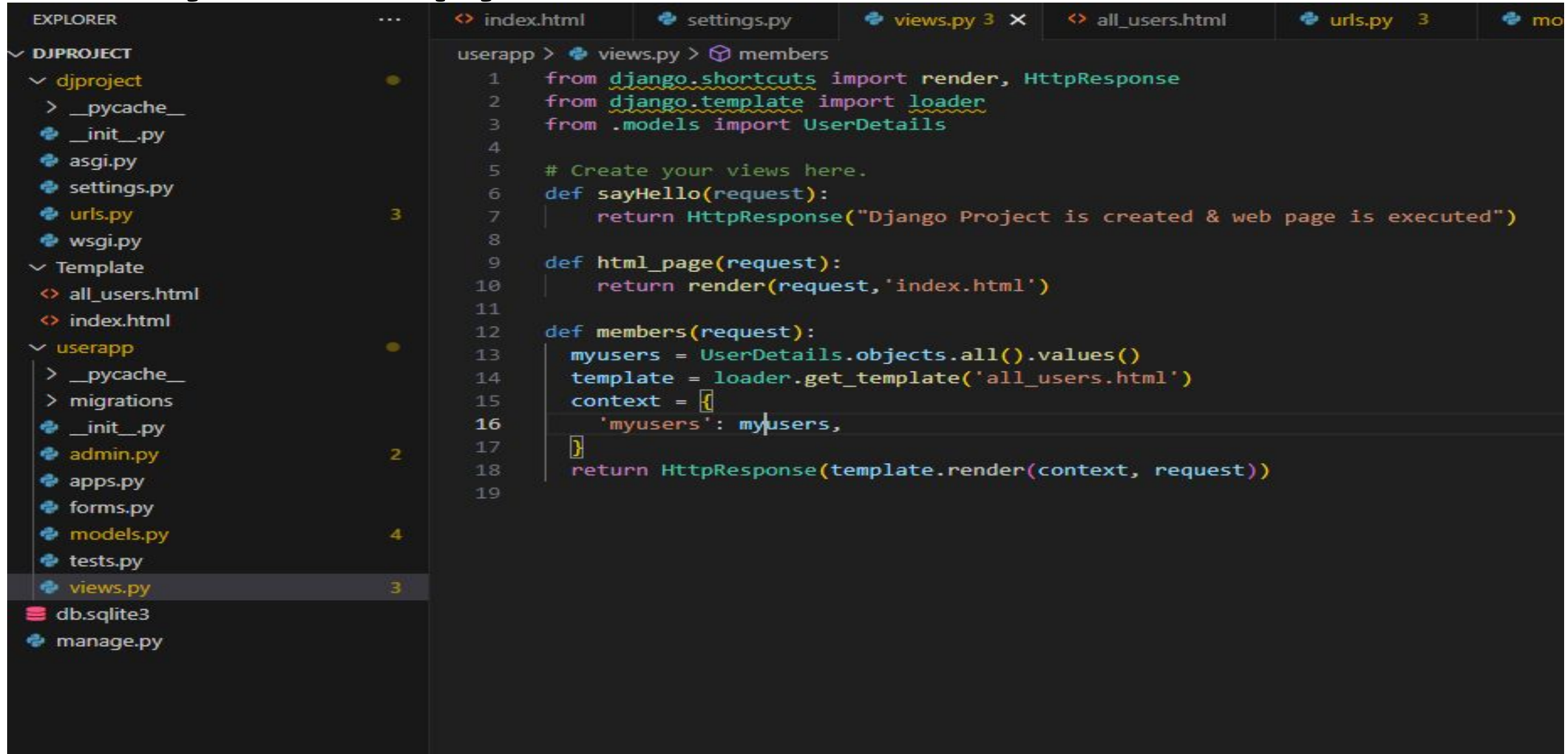
all_users.html:

```

1  <html>
2  <body>
3
4  <h1>Members</h1>
5
6  <ul>
7      {% for x in myusers %}
8          <li>{{ x.firstname }} {{ x.lastname }}</li>
9      {% endfor %}
10 </ul>
11
12 </body>
13 </html>
  
```

- `{% %}` brackets inside the HTML document are Django Tags, telling Django to perform some programming logic inside these brackets.

Modify views.py



The screenshot shows a code editor with a dark theme. On the left is the 'EXPLORER' sidebar showing the project structure. The main editor area displays the 'views.py' file for the 'userapp' app, with the 'members' view function selected.

EXPLORER Sidebar:

- DJPROJECT
 - ▼ dproject
 - > __pycache__
 - __init__.py
 - asgi.py
 - settings.py
 - urls.py (3)
 - wsgi.py
 - ▼ Template
 - <> all_users.html
 - <> index.html
 - ▼ userapp
 - > __pycache__
 - > migrations
 - __init__.py
 - admin.py (2)
 - apps.py
 - forms.py
 - models.py (4)
 - tests.py
 - views.py (3)
 - db.sqlite3
 - manage.py

Main Editor (views.py):

```

userapp > views.py > members
1  from django.shortcuts import render, HttpResponseRedirect
2  from django.template import loader
3  from .models import UserDetails
4
5  # Create your views here.
6  def sayHello(request):
7      return HttpResponseRedirect("Django Project is created & web page is executed")
8
9  def html_page(request):
10     return render(request, 'index.html')
11
12  def members(request):
13     myusers = UserDetails.objects.all().values()
14     template = loader.get_template('all_users.html')
15     context = {}
16     'myusers': myusers,
17
18     return HttpResponseRedirect(template.render(context, request))
19
  
```

The members view does the following:

- Creates a UserDetails object with all the values of the Member model.
- Loads the all_users.html template.
- Creates an object containing the myuser object.
- Sends the object to the template.
- Outputs the HTML that is rendered by the template.

Update urls.py

The screenshot shows a code editor with a dark theme. On the left is the 'EXPLORER' sidebar showing the project structure. The main editor area displays the contents of 'urls.py'.

EXPLORER:

- DJPROJECT
 - djproject
 - __pycache__
 - __init__.py
 - asgi.py
 - settings.py
 - urls.py** (3)
 - wsgi.py
 - Template
 - all_users.html
 - index.html
 - userapp
 - __pycache__
 - migrations
 - __init__.py
 - admin.py (2)
 - apps.py
 - forms.py
 - models.py (4)
 - tests.py
 - views.py (3)
 - db.sqlite3
 - manage.py

urls.py:

```

1  """
2  URL configuration for djproject project.
3
4  The `urlpatterns` list routes URLs to views. For more information please see:
5  |     https://docs.djangoproject.com/en/4.2/topics/http/urls/
6  | Examples:
7  | Function views
8  |     1. Add an import:  from my_app import views
9  |     2. Add a URL to urlpatterns:  path('', views.home, name='home')
10 | Class-based views
11 |     1. Add an import:  from other_app.views import Home
12 |     2. Add a URL to urlpatterns:  path('', Home.as_view(), name='home')
13 | Including another URLconf
14 |     1. Import the include() function: from django.urls import include, path
15 |     2. Add a URL to urlpatterns:  path('blog/', include('blog.urls'))
16 | """
17 from django.contrib import admin
18 from django.urls import path
19 from userapp import views
20
21 urlpatterns = [
22     path('admin/', admin.site.urls),
23     path('html/', views.html_page),
24     path('users/', views.members),
25 ]
26
27
  
```

Run server (python manage.py runserver)



Members

- Reena Singh
- Janvi Singh

*Thank
you*